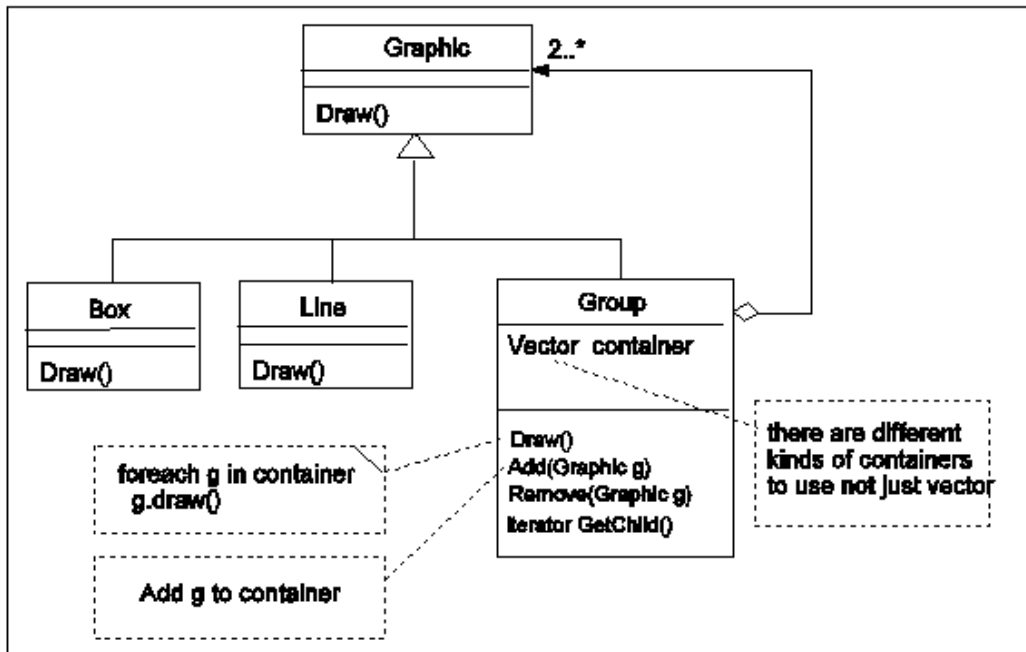


(3 倍分數)

當你為 UML editor 增加一個 Group 的物件並且把其行為(methods) 逐漸地加進去。如下圖一。其中 add(), remove(), GetChild() 都是 Group 的特異化行為。所以只存在於 Group 物件。



(圖一)

假設我們在 Canvas 用了一個 vector AllObjectList 來存放所有在編輯區的物件
另外假設目前有 A,B,C,D 四個物件。

1. 一開始四個物件平行的存放於 AllObjectList 如下圖

A B C D

2. 假設 物件 B, C 如今被 group 成 Group 物件,

λ 我們會先建立一個 group 物件 G1

λ 將 B, C 從 AllObjectList 移除

λ 呼叫 G.add(B), G.add(C)

λ 最後再把物件 G 加入到 AllObjectList 中

最後的狀態如下圖二



(圖二)

所有落在 B, C 的 mouse 事件, 改由 G1 接收, B,C draw()的功能也改由 G1 去進行呼叫。到目前為止好像沒有問題, 一切圓滿。

但是問題來了, 譬如你們的 umleditor usecase D.2 要去 ungroup 一個物件。假設使用者事先點選了某一個物件, 然後到 Edit Menu 選擇 ungroup 的功能。若被點選的物件記錄在 selected 這個變數, 這時候你不得不寫下下面的一段 code

```
MENU_EDITactionperformed() {  
    if (selected instanceof Group)  
        // 我們要把 selected 的所有 child 物件移除, 再加入 AllObjectList  
        for all child c in selected.getChild()  
            selected.remove(c);  
            add c to AllObjectList ;  
        delete selected  
    } else {  
        // do nothing  
    }  
}
```

也就是說, 我們必須先判斷 selected 是否是 Group 物件, 如果是, 我們才能呼叫 group 物件所特有的 getChild(), remove() ...等等呼叫

這樣必須要去區分是否為 group object 的例子也會出現在其他地方。當出現的地方一多的時候會對 programmer 開始造成困擾。Programmer 不能專心的乾淨的解決一件事情, 往往還要擔心如果物件型別不一樣的時候, 要做什麼特殊的處理。

請問解決的方法是?

A:

本題可以使用 **Composite Pattern** 解決。因為 Box、Line、Group 都可以被視為一種 Graphic，所以應該設計一個共同的父類別或介面 Graphic，讓所有物件都繼承它。Graphic 裡面定義共同的操作，例如 draw()，也可以定義 add()、remove()、getChild() 或 ungroup() 等方法。對於 Box、Line 這種 leaf object，這些 group 相關方法可以預設為 do nothing；對於 Group 這種 composite object，則真正實作這些方法，並用一個 container 儲存它的 child objects。

如此一來，client 端或 menu action 就不需要一直判斷 selected instanceof Group。當使用者選擇 ungroup 時，只要呼叫：

```
selected.ungroup(canvas);
```

程式就會依照物件本身的型別自動執行正確行為。若 selected 是 Group，就展開其 child objects 並放回 AllObjectList；若 selected 是 Box 或 Line，就不做任何事。這樣可以讓程式碼更乾淨，減少型別判斷，也符合物件導向的 polymorphism 精神。